Stack Data Structure Study Guide

1. Introduction to Stack

What is a Stack?

A Stack is a linear data structure that follows the **Last In, First Out (LIFO)** principle. This means the last element added to the stack is the first one to be removed. Think of it like a stack of plates - you can only add or remove plates from the top.

Key Characteristics

- **LIFO Principle**: Last In, First Out
- **Single Access Point**: Elements can only be added or removed from the top
- **Dynamic Size**: Can grow and shrink during runtime

Basic Stack Operations

Push Operation

Adds an element to the top of the stack.

```java
Stack<Integer> stack = new Stack<>();
stack.push(10);
stack.push(20);
stack.push(30);
// Stack now contains: [10, 20, 30] (30 is at the top)
```

Pop Operation

Removes and returns the top element from the stack.

```java
int topElement = stack.pop(); // Returns 30
// Stack now contains: [10, 20]
```

Peek/Top Operation

Returns the top element without removing it.

```java
int topElement = stack.peek(); // Returns 20 (doesn't remove it)
// Stack still contains: [10, 20]
```

isEmpty Operation

Checks if the stack is empty.

```java
boolean empty = stack.isEmpty(); // Returns false
```

Size Operation

Returns the number of elements in the stack.

```java
int size = stack.size(); // Returns 2
```

Stack Implementation in Java

Using Built-in Stack Class

```java
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        // Push elements
        stack.push(5);
        stack.push(10);
        stack.push(15);

        // Display top element
        System.out.println("Top element: " + stack.peek());

        // Pop elements
        while (!stack.isEmpty()) {
            System.out.println("Popped: " + stack.pop());
        }
    }
}
```

Array-based Stack Implementation

```java
public class ArrayStack {
    private int[] stack;
    private int top;
    private int capacity;

    public ArrayStack(int size) {
        stack = new int[size];
        capacity = size;
        top = -1;
    }

    public void push(int item) {
        if (top == capacity - 1) {
            System.out.println("Stack Overflow");
            return;
        }
        stack[++top] = item;
    }

    public int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
            return -1;
        }
        return stack[top--];
    }

    public int peek() {
        if (top == -1) {
            System.out.println("Stack is empty");
            return -1;
        }
        return stack[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }
}
```

2. Application and Usage

Common Applications of Stack

Function Call Management

Stacks are used to manage function calls in programming languages. When a function is called, its variables and return address are pushed onto the call stack.

```java
public class RecursionExample {
    public static int factorial(int n) {
        if (n <= 1) {
            return 1;
        }
        return n * factorial(n - 1); // Each call is pushed onto the stack
    }
}
```

Parentheses Matching

Checking if parentheses, brackets, and braces are properly balanced.

```java
import java.util.Stack;

public class ParenthesesChecker {
    public static boolean isBalanced(String expression) {
        Stack<Character> stack = new Stack<>();

        for (char ch : expression.toCharArray()) {
            if (ch == '(' || ch == '[' || ch == '{') {
                stack.push(ch);
            } else if (ch == ')' || ch == ']' || ch == '}') {
                if (stack.isEmpty()) {
                    return false;
                }
                char top = stack.pop();
                if (!isMatchingPair(top, ch)) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }

    private static boolean isMatchingPair(char opening, char closing) {
        return (opening == '(' && closing == ')') ||
               (opening == '[' && closing == ']') ||
               (opening == '{' && closing == '}');
    }
}
```

Undo Operations

Many applications use stacks to implement undo functionality.

```java
import java.util.Stack;

public class UndoRedoExample {
    private Stack<String> undoStack = new Stack<>();
    private Stack<String> redoStack = new Stack<>();
    private String currentState = "";

    public void performAction(String newState) {
        undoStack.push(currentState);
        currentState = newState;
        redoStack.clear(); // Clear redo stack when new action is performed
    }

    public void undo() {
        if (!undoStack.isEmpty()) {
            redoStack.push(currentState);
            currentState = undoStack.pop();
        }
    }

    public void redo() {
        if (!redoStack.isEmpty()) {
            undoStack.push(currentState);
            currentState = redoStack.pop();
        }
    }
}
```

Browser History

Web browsers use stacks to implement the back button functionality.

Expression Evaluation

Stacks are essential for evaluating mathematical expressions, especially when converting between different notations.

3. Prefix, Infix and Postfix

Understanding Expression Notations

Infix Notation

The operator is placed between operands. This is the most common notation we use in everyday mathematics.

- Example: `A + B * C`
- Example: `(A + B) * C`

Prefix Notation (Polish Notation)

The operator is placed before the operands.

- Example: `+ A * B C` (equivalent to `A + (B * C)`)
- Example: `* + A B C` (equivalent to `(A + B) * C`)

Postfix Notation (Reverse Polish Notation)

The operator is placed after the operands.

- Example: `A B C * +` (equivalent to `A + (B * C)`)
- Example: `A B + C *` (equivalent to `(A + B) * C`)

Converting Infix to Postfix

java

```java
import java.util.Stack;

public class InfixToPostfix {

    private static int precedence(char operator) {
        switch (operator) {
            case '+':
            case '-':
                return 1;
            case '*':
            case '/':
                return 2;
            case '^':
                return 3;
            default:
                return -1;
        }
    }

    public static String infixToPostfix(String infix) {
        StringBuilder result = new StringBuilder();
        Stack<Character> stack = new Stack<>();

        for (char ch : infix.toCharArray()) {
            if (Character.isLetterOrDigit(ch)) {
                result.append(ch);
            } else if (ch == '(') {
                stack.push(ch);
            } else if (ch == ')') {
                while (!stack.isEmpty() && stack.peek() != '(') {
                    result.append(stack.pop());
                }
                stack.pop(); // Remove '('
            } else { // Operator
                while (!stack.isEmpty() &&
                       precedence(ch) <= precedence(stack.peek())) {
                    result.append(stack.pop());
                }
                stack.push(ch);
            }
        }

        while (!stack.isEmpty()) {
```

```
            result.append(stack.pop());
        }

        return result.toString();
    }
}
```

Evaluating Postfix Expressions

```java
import java.util.Stack;

public class PostfixEvaluator {

    public static int evaluatePostfix(String postfix) {
        Stack<Integer> stack = new Stack<>();

        for (char ch : postfix.toCharArray()) {
            if (Character.isDigit(ch)) {
                stack.push(ch - '0'); // Convert char to int
            } else {
                int operand2 = stack.pop();
                int operand1 = stack.pop();

                switch (ch) {
                    case '+':
                        stack.push(operand1 + operand2);
                        break;
                    case '-':
                        stack.push(operand1 - operand2);
                        break;
                    case '*':
                        stack.push(operand1 * operand2);
                        break;
                    case '/':
                        stack.push(operand1 / operand2);
                        break;
                }
            }
        }

        return stack.pop();
    }
}
```

Converting Infix to Prefix

```java
import java.util.Stack;

public class InfixToPrefix {

    public static String infixToPrefix(String infix) {
        // Step 1: Reverse the infix expression
        StringBuilder reversed = new StringBuilder(infix).reverse();

        // Step 2: Replace '(' with ')' and vice versa
        for (int i = 0; i < reversed.length(); i++) {
            if (reversed.charAt(i) == '(') {
                reversed.setCharAt(i, ')');
            } else if (reversed.charAt(i) == ')') {
                reversed.setCharAt(i, '(');
            }
        }

        // Step 3: Convert to postfix
        String postfix = InfixToPostfix.infixToPostfix(reversed.toString());

        // Step 4: Reverse the result
        return new StringBuilder(postfix).reverse().toString();
    }
}
```

4. Practice Questions

Stack Traversal and Display

```java
import java.util.Stack;

public class StackTraversal {

    public static void displayStack(Stack<Integer> stack) {
        if (stack.isEmpty()) {
            System.out.println("Stack is empty");
            return;
        }

        System.out.print("Stack (top to bottom): ");
        Stack<Integer> temp = new Stack<>();

        // Pop all elements to temp stack
        while (!stack.isEmpty()) {
            int element = stack.pop();
            System.out.print(element + " ");
            temp.push(element);
        }

        // Restore original stack
        while (!temp.isEmpty()) {
            stack.push(temp.pop());
        }
        System.out.println();
    }
}
```

Finding Minimum Element in Stack

```java
import java.util.Stack;

public class MinStack {
    private Stack<Integer> stack;
    private Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int value) {
        stack.push(value);

        if (minStack.isEmpty() || value <= minStack.peek()) {
            minStack.push(value);
        }
    }

    public int pop() {
        if (stack.isEmpty()) {
            return -1;
        }

        int popped = stack.pop();
        if (popped == minStack.peek()) {
            minStack.pop();
        }
        return popped;
    }

    public int getMin() {
        if (minStack.isEmpty()) {
            return -1;
        }
        return minStack.peek();
    }
}
```

Reversing a String using Stack

```java
import java.util.Stack;

public class StringReverser {

    public static String reverseString(String str) {
        Stack<Character> stack = new Stack<>();

        // Push all characters to stack
        for (char ch : str.toCharArray()) {
            stack.push(ch);
        }

        // Pop all characters to form reversed string
        StringBuilder reversed = new StringBuilder();
        while (!stack.isEmpty()) {
            reversed.append(stack.pop());
        }

        return reversed.toString();
    }
}
```

Quiz Questions

1. What principle does a Stack data structure follow? a) First In, First Out (FIFO) b) Last In, First Out (LIFO) c) Random Access d) Sequential Access

Answer: b) Last In, First Out (LIFO)

2. Which operation is used to add an element to the top of a stack? a) pop() b) peek() c) push() d) insert()

Answer: c) push()

3. What is the postfix notation for the infix expression "A + B * C"? a) A B C * + b) + A * B C c) A B + C * d) A + B C *

Answer: a) A B C * +

4. What happens when you try to pop an element from an empty stack? a) Returns null b) Returns 0 c) Throws an exception or shows stack underflow d) Nothing happens

Answer: c) Throws an exception or shows stack underflow

5. In which of the following applications are stacks commonly used? a) Function call management b) Undo operations in text editors c) Expression evaluation d) All of the above

Answer: d) All of the above

Practice Coding Questions

1. Implement a method to check if a given string of parentheses is balanced. The string may contain '(', ')', '[', ']', '{', '}'. Return true if balanced, false otherwise.

2. Write a program to reverse the elements of a stack without using any additional data structures (like arrays or other stacks).

**3. Create a method that converts a given infix expression to its equivalent postfix expression. Handle operators +, -, , /, and parentheses.*

4. Implement a stack that supports push, pop, and getMin operations, all in O(1) time complexity.

**5. Write a program to evaluate a postfix expression. The expression contains single-digit numbers and operators +, -, , /.*